

PRUEBA TÉCNICA · CIBERSEGURIDAD

Reporte de Uso de IA

Transparencia, herramientas y verificación

PREPARADO POR	Lady Marcela Romero Rivero
ROL	Líder de Ciberseguridad
CLIENTE	Simon Movilidad
ESCENARIO	FleetSec S.A.S. — Telemetría de flotas
FECHA	25 de agosto de 2026
VERSIÓN	1.0

CONFIDENCIAL — Exclusivo para Simon Movilidad

Tabla de contenido

9. Herramientas y tareas concretas donde se usó IA

**10. Alucinación / error de seguridad generado por la IA y cómo se corrigió
(OBLIGATORIO)**

11. Tareas que NO delegaría a IA sin supervisión (máx. 150 palabras)

Este proyecto se construyó con asistencia de IA (Claude) bajo supervisión y verificación humana. Se documenta de forma específica y honesta.

9. Herramientas y tareas concretas donde se usó IA

Tarea	Cómo se usó la IA	Verificación humana aplicada
App vulnerable + parches (10 CWE)	Generación del código / <code>vuln</code> y <code>/secure</code> en Express	Se ejecutaron los 20 tests (<code>node --test</code>) — 20/20 en verde antes de aceptar
Reglas Semgrep propias	Redacción inicial de patrones	Se ejecutó Semgrep contra el código; se corrigieron patrones que no disparaban (paso a modo <i>taint</i>)
Módulo Terraform	Generación de recursos AWS	<code>checkov</code> real (251 passed / 0 failed) + parseo HCL de los 12 archivos
Reglas Sigma	Redacción a partir de la línea de tiempo	Validación con <code>pysigma</code> (<code>SigmaCollection.from_yaml</code>) — se corrigió un modificador inválido
CVSS de los 10 hallazgos	Propuesta de vectores	Recalculados con una calculadora CVSS 3.1 propia; se ajustaron los que no cuadraban
Playbook IR / comandos AWS CLI	Redacción de pasos	Revisión de reversibilidad y orden (evidencia antes de contención)
Informe VAPT en PDF	Render Markdown→PDF (weasyprint)	Inspección visual del PDF

10. Alucinación / error de seguridad generado por la IA y cómo se corrigió (OBLIGATORIO)

Sí hubo errores. Los más relevantes:

- Digest SHA256 inventado en el Dockerfile (alucinación clásica).** La primera versión del `Dockerfile` fijaba la imagen base con `node:20-bookworm-slim@sha256:2f7ceb6f...` — un hash *plausible pero completamente inventado*. De haberse confiado, `docker build` habría fallado (o peor, en un registry mirror podría haber apuntado a una imagen incorrecta). **Detección:** un digest legítimo solo puede obtenerse de `docker inspect / docker pull`, nunca escribirse de memoria. **Corrección:** se cambió a un tag pinneado real (`node:20.18.1-bookworm-slim`) y se documentó el procedimiento exacto para endurecer a digest immutable con `docker inspect --format='{{index .RepoDigests 0}}'`.
- Reglas Semgrep que no disparaban.** Los patrones AST iniciales (`db.exec(\ ...`)` ) no cubrían el caso real (variable intermedia). **Detección:** al ejecutar Semgrep arrojó 0 findings. **Corrección:** se reescribieron en modo *taint* ( `source req.query.* → sink db.exec / fetch` ), verificando que disparan en `/vuln` y **no** en `/secure` (0 falsos positivos).
- Modificador Sigma inexistente (`nocase`).** Provenía de una plantilla. **Detección:** `pysigma` lanzó `Unknown modifier 'nocase'`. **Corrección:** se eliminó (el matching por defecto ya es case-insensitive en la mayoría de backends).
- Check Checkov personalizado en Python que no cargaba** en esta versión del runner. **Detección:** pruebas empíricas mostraron que ni un check trivial externo en Python se

ejecutaba. **Corrección honesta:** la regla de puertos administrativos queda **enforced por los checks nativos CKV_AWS_24/25** (probados: pasan mis SG y fallan un SG malo), y se añadió una política YAML (**CKV2_FLEETSEC_1**) + un self-test unitario del Python. No se ocultó la limitación.

Conclusión: la IA acelera, pero **cada artefacto se validó con una herramienta real** (tests, Semgrep, Checkov, pysigma, calculadora CVSS). La verificación empírica —no la confianza— es lo que hizo el trabajo utilizable.

11. Tareas que NO delegaría a IA sin supervisión (máx. 150 palabras)

No delegaría sin revisión humana: (1) la **respuesta a incidentes en vivo** —contener, revocar credenciales o aislar instancias mal puede destruir evidencia o tumbar producción; (2) la **aceptación o clasificación final de riesgo** y las decisiones de cumplimiento (p. ej. si un incidente gatilla notificación a la SIC bajo Ley 1581), porque implican criterio legal y de negocio; (3) la **rotación/manejo de secretos y llaves** reales en producción; (4) la **aprobación de excepciones de seguridad** (supresiones, break-glass) —deben tener dueño humano y trazabilidad; (5) **payloads de explotación contra sistemas de terceros** sin autorización explícita. La IA es excelente para generar borradores, detección y automatización, pero el criterio, la rendición de cuentas y las acciones irreversibles deben quedar en manos de un ingeniero responsable. La IA también puede alucinar (ver §10): todo output se verifica.